

9. Analisi ammortizzata e problemi Union-Find



Lezione 18 - @November 3, 2025

Analisi ammortizzata

Quanto costa incrementare un contatore a n bit

costo = numero di bit da ispezionare

0 0 0 1 0 1 1 1

1 1 1 1 1 1 1 1

0 0 0 1 1 0 0 0

0 0 0 0 0 0 0 0

```
ALGORITMO incrementaContatore(Array v[0...n-1] di bit)
  i <- 0
  WHILE i < n AND v[i] = 1 DO
    v[i] <- 0
    i <- i + 1
  IF i < n THEN v[i] <- 1
```

- Costo caso peggiore = $O(n)$
- costo di k operazioni di incremento sullo stesso contatore $O(k \cdot n) \leftarrow$ eccessivo!

Esempio

Iterazione	Valore	#bit modificati
0	0 0 0 0 0 0 0	1
1	0 0 0 0 0 1	1
2	0 0 0 0 1 0	2
3	0 0 0 0 1 1	1
4	0 0 0 1 0 0	3
5	0 0 0 1 0 1	1
6	0 0 0 1 1 0	2
7	0 0 0 1 1 1	1
8	0 0 1 0 0 0	4

- il bit 0 cambia ogni volta
- il bit 1 cambia ogni 2 volte
-
- il bit i cambia ogni 2^i volte

Sia $T(n, k)$: numero di operazioni per incrementare k volte un contatore a n bit che parte da 0.

$$\begin{aligned}
T(n, k) &= k + \left\lfloor \frac{k}{2} \right\rfloor + \left\lfloor \frac{k}{2^2} \right\rfloor + \dots + \left\lfloor \frac{k}{2^{n-1}} \right\rfloor \\
&= \sum_{i=0}^{n-1} \left\lfloor \frac{k}{2^i} \right\rfloor \leq \sum_{i=0}^{n-1} \frac{k}{2^i} = k \sum_{i=0}^{n-1} \frac{1}{2^i} \\
&\leq k \sum_{i=0}^{\infty} \frac{1}{2^i} = k \sum_{i=0}^{\infty} \left(\frac{1}{2} \right)^i =^* k \frac{1}{1 - \frac{1}{2}} = 2k
\end{aligned}$$

$$* \text{ serie geometrica } \sum_{i=0}^{\infty} \alpha^i = \frac{1}{1 - \alpha}$$

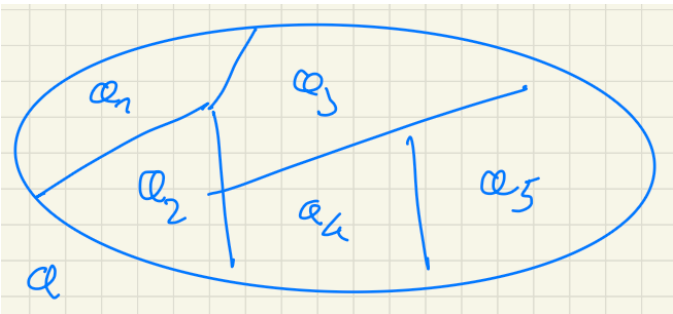
$$T_{a,n}(k) = \frac{T(n, k)}{k} = \frac{2k}{k} = 2 \rightarrow \text{costo ammortizzato}$$

Le operazioni precedenti influenzano il costo delle operazioni successive.

Rappresentazione di partizioni

anche dette **union/find**

Dato un insieme di insieme $\mathcal{A}$



Una partizione di $\mathcal{A}$:

Famiglia di sottoinsiemi $a_1, a_2, \dots, a_k \subseteq A$ tale che:

- per $i = 1, \dots, k$ $a_i \neq \emptyset$
- per $i, j = 1, \dots, k$ se $i \neq j \Rightarrow a_i \cap a_j = \emptyset$
- $\bigcup_{i=1}^k a_i = A$

$a_1, a_2, \dots, a_k$ si chiamano parti

Sia $\mathcal{S}$ collezione di elementi tra loro distinti

esempio interi di 1 a n

- organizzazione iniziale
 - n insiemi disgiunti - $\{1\}, \{2\}, \dots, \{n\}$
- ogni insieme ha un nome
 - nome iniziale - $\{i\} \rightarrow i$

Problema union-find

Rappresentare una collezione di insiemi disgiunti, con le operazioni:

- **UNION(A, B)** unisce gli insiemi A e B in un unico insieme di nome di nome A
- **FIND(x)** restituisce il nome dell'insieme che contiene l'elemento **x**
- **MAKESET(x)** crea un nuovo insieme **{x}** di nome **x** con **x** nuovo elemento

Vari tipi di soluzione

- soluzioni elementari
- soluzioni evolute

In tutte le soluzioni:

- ogni insieme (parte) è rappresentato da un albero con radice
 - odi → elementi dell’insieme
 - radice → nome dell’insieme
- implementazione attraverso *puntatori verso l’alto*
- partizione: foreste di alberi

Algoritmi “QuickFind”

- alberi di altezza 1
- elementi dell’insieme ↔ **foglie**
- nome dell’insieme ↔ **radice**

Operazioni

operazione	tempo - $T(n)$	spazio
MAKESET(elemento e)	$O(1)$	
FIND(elemento e)	$O(1)$	
UNION(nome A, nome B)	$O(n)$	$O(n)$

QuickFind con bilanciamento

```
UNION(nome A, nome B)
  Quando #B > #B
    anziché collegari gli elementi di B sotto A
    conviene collegare gli elementi di A sotto B
    e rinominare la radice
```

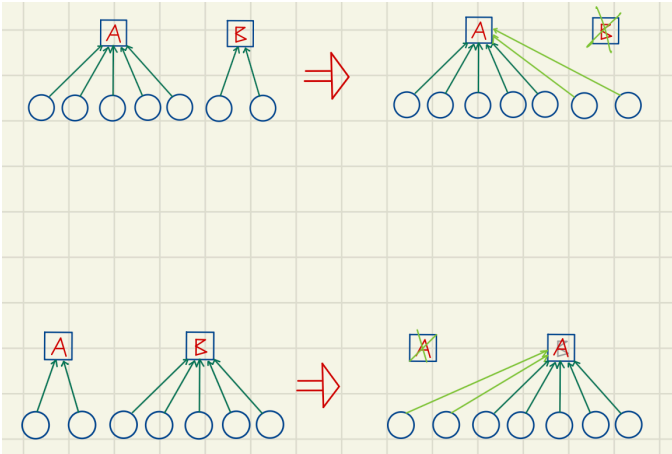
Caso peggiore $\frac{n}{2}$ nodi in A e $\frac{n}{2}$ nodi in B

$$T(n) = O(n)$$

Operazioni

Nella radice si memorizza il numero di elementi dell’insieme (**SIZE**)

```
UNION(nome A, nome B)
  IF size(A) >= size(B) THEN
    sposta i puntatori delle foglie d
  i B ad A
    rimuovi la radice B
  ELSE
    sposta i puntatori dalle foglie d
  i A a B
    rinomina la radice di B a B
    rimuovi la vecchia radice A
    size(A) <- size(A) + size(B)
```



Si può dimostrare che effettuando una sequenza di:

- n MAKESET
- $O(n)$ UNION e FIND

il tempo totale è $O(n \log n)$

→ costo ammortizzato UNION $O(\log n)$

Algoritmi Quick Union

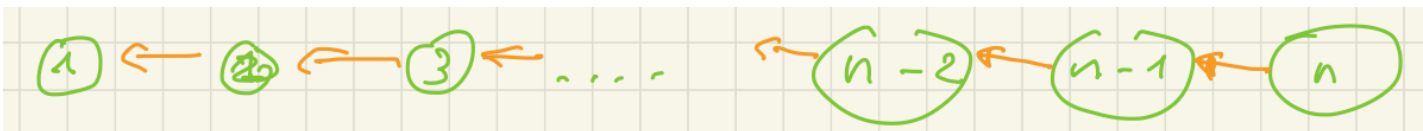
- Alberi di varie altezze
- elementi dell'insieme ↔ **nodi**
- elemento che da il nome all'insieme ↔ **radice**

Operazioni

operazione	tempo - $T(n)$	note
<code>MAKESET(elemento e)</code>	$T(n) = O(1)$	
<code>UNION(nome A, nome B)</code>	$T(n) = O(1)$	
<code>FIND(elemento e) -> nome</code>	$T(n)$	risalire da e fino alla radice

Nota: albero peggiore

`MAKESET(1), MAKESET(2), ..., MAKESET(n)`

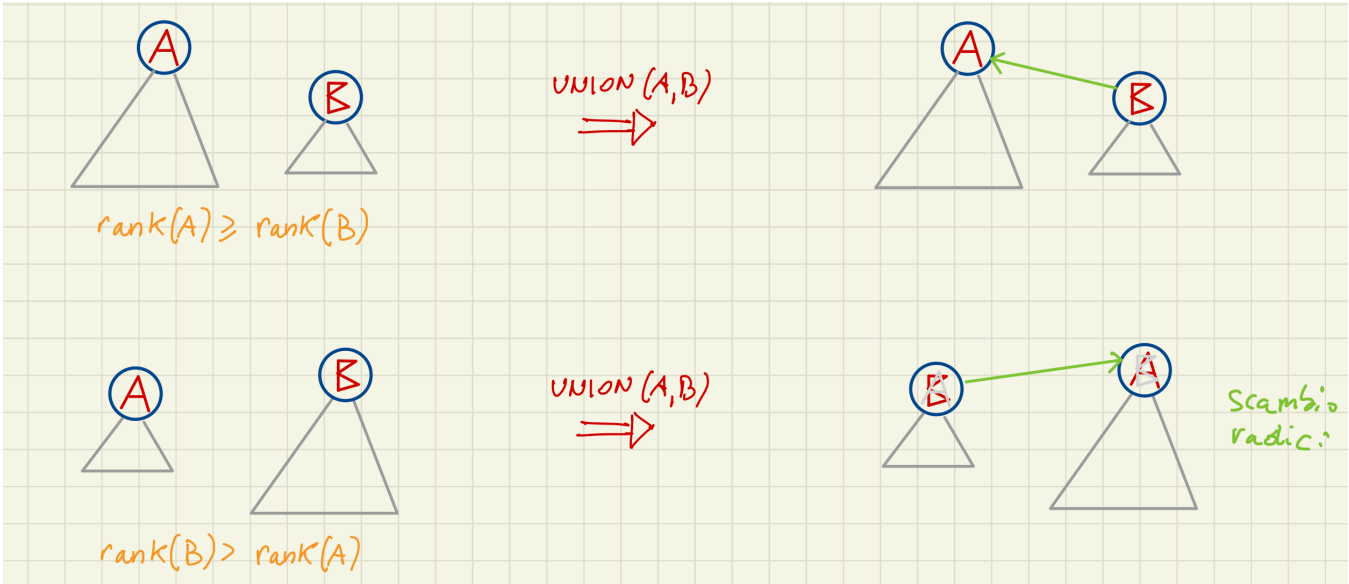


→ Albero di altezza $n - 1$

Union bilanciata

attacciamo la radice dell'albero più basso sotto quella dell'albero più alto

Sia rank = altezza albero



Lezione 19 - @November 5, 2025

puntatori verso l'alto.

Union **by rank**

nella radice si memorizza l'altezza dell'albero (rank)

```
UNION(nome A, nome B)
  IF rank(A) > rank(B) THEN
    rendi la radice di B figlia di quella di A
  ELSE IF rank(B) > rank(A) THEN
    rendi la radice di A figlia di quella di B
    scambia i contenuti dei nodi A e B
  ELSE
```

```
rendi la radice di B figlia di quella di A
rank(A) <- rank(A) + 1
```

$$T(n) = O(1)$$

Le altre operazioni

- nell'operazione di `makeset` occorre memorizzare nella radice che il rank è 0
- nell'operazione `find`, occorre risalire nell'albero, il tempo dunque dipende dall'altezza dell'albero:

»

Lemma

ogni albero "quick union" costruito effettuando operazioni di Union bilanciate contiene almeno $2^{\text{rank}(x)}$, con x radice

$$\# \text{nodi}(x) \geq 2^{\text{rank}(x)}$$

dim: sia k il numero di operazioni di `union` con cui l'albero con radice x è stato ottenuto

induzione su k

- base: $k = 0$:

$$\text{radice } x \rightarrow \# \text{nodi}(x) = 1 \quad \text{rank}(x) = 0 \quad 1 \geq 2^0$$

- passo: $< k \rightarrow k$:

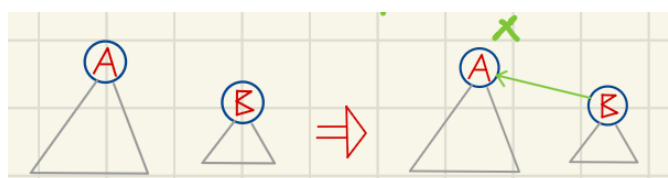
Sia `UNION(A, B)` l'ultima operazione di union con cui è stato ottenuto l'albero di radice x

Gli alberi con radice A e radice B, sono stati ottenuto con meno di k union

$$\Rightarrow \text{ip. induzione} \rightarrow \# \text{nodi}(A) \geq 2^{\text{rank}(A)} \text{ e } \# \text{nodi}(B) \geq 2^{\text{rank}(B)}$$

tre casi:

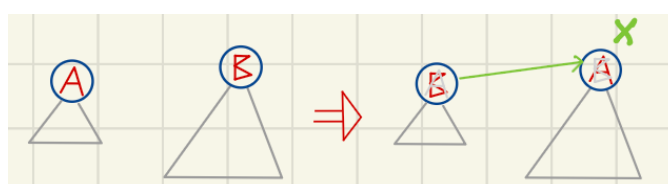
1. Se $\text{rank}(A) > \text{rank}(B) \rightarrow \text{rank}(x) = \text{rank}(A)$



$$\begin{aligned} \# \text{nodi}(x) &= \# \text{nodi}(A) + \# \text{nodi}(B) \\ &\geq \# \text{nodi}(A) \geq 2^{\text{rank}(A)} = 2^{\text{rank}(x)} \end{aligned}$$

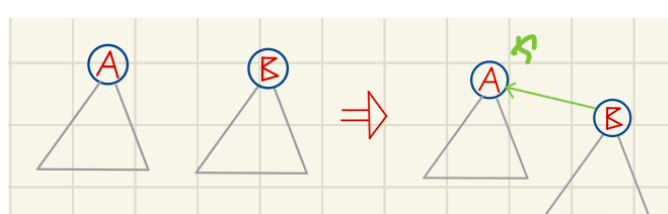
$$\Rightarrow \# \text{nodi}(x) \geq 2^{\text{rank}(x)}$$

2. Se $\text{rank}(A) < \text{rank}(B) \rightarrow \text{rank}(x) = \text{rank}(B)$



come prima, mettendo B al posto di A

3. Se $\text{rank}(A) = \text{rank}(B) \rightarrow \text{rank}(x) = \text{rank}(A) + 1$



$$\begin{aligned} \# \text{nodi}(x) &= \# \text{nodi}(A) + \# \text{nodi}(B) \\ &\geq 2^{\text{rank}(A)} + 2^{\text{rank}(B)} \\ &= 2 \cdot 2^{\text{rank}(A)} = 2^{\text{rank}(A)+1} = 2^{\text{rank}(x)} \end{aligned}$$

$$\Rightarrow \# \text{nodi}(x) \geq 2^{\text{rank}(x)}$$

otteniamo che

$$\text{rank}(x) \leq \log_2 \# \text{nodi}(x)$$

se ci sono n elementi l'altezza è al massimo $\log_2 n$

costo `find` $T(n) = O(\log n)$

Ricapitolando `union find`

	MAKESET	UNION	FIND
quickFind	$O(1)$	$O(n)$	$O(1)$
quickFind bilanciata	$O(1)$	$O(\log n)$ ammortizzato	$O(1)$
quickUnion	$O(1)$	$O(1)$	$O(n)$
quickUnion bilanciata	$O(1)$	$O(1)$	$O(\log n)$
quickUnion bilanciata con compressione di cammino	$O(1)$	$O(1)$	$O(\log^* n)$

Quick union bilanciata con compressione di cammino

Si può dimostrare che effettuando una sequenza di:

- n `makeset`
- $O(n)$ `union` e `find`

il tempo totale è $O(n \log^* n)$

$\Rightarrow$ costo ammortizzato di `FIND` è $O(\log^* n)$

Logaritmo iterato

$$\log^{(i)} n = \log_2 \log_2 \dots \log_2 n$$

i volte

$$\log^* n = \min\{i \mid \log^{(i)} n \leq 1\}$$

è una funzione che va all'∞ ma in maniera MOLTO LENTA, per i valori rappresentabili sui computer si può assumere che i valori siano compresi tra 5 e 6